

Custom SQL Journey Documentation

Awwab Wadekar

July 21, 2026

1 Introduction

I was bored in my vacation, so I thought why not learn how SQL works under the hood with an attempt on a custom implementation?

This is torture. It was not fun.

Anyways the journey is a 5 phase mess:

Phase 1. The OS background

Phase 2. The Volcano Model

Phase 3. The System Catalog

Phase 4. The Parser Dilemma

Phase 5. The Editor Interface

2 Phase 1: The OS background

2.1 Day 1: Serialisation and Deserialisation

Data is often stored in Raw Bytes in Memory. So when we fetch it we actually can't use something like `struct.id` or `struct.name` to access fields.

Instead we use an offset variable to keep track of what we have accessed.

```
struct MyStruct{
    char name[30];
    int id;
}
```

Now here the total memory is $30 * 1 + 4 = 34$ bytes, as one char is 1 byte and one int is 4 bytes.

But usually the OS assigns Memory in terms of Pages, which here are considered to be 4 bytes in size. So you have Page 1...Page 2...Page 3.....Page 8...Page 9. Basically $9 * 4 = 36$ bytes.

Wait a minute, what ? I'm not a Mathematics Graduate but surely $36 > 34$.

So what actually happened here is padding, in the 8th Page, 2 bytes are used for the last 2 elements of the char array.

But we cant exactly put 2 bytes of an int in one page and 2 in another. And we for sure cannot have half a page. So the OS simply pads those 2 bytes up.

Well how do we fix this? The solution : Interpret everything as raw bytes and track offset of every field. Something along the lines of :

```
vector<uint8_t> buffer;
size_t offset = 0;
MyStruct obj;
obj.name = "Awwab";
obj.id = 77;
std::memcpy(buffer.data() + offset, &obj.name,
    sizeof(obj.name))
offset += sizeof(obj.name);
std::memcpy(buffer.data() + offset, &obj.id,
    sizeof(obj.id))
```

```
offset += sizeof(obj.id);
```

Similarly, Deserialization does the same process, just this time the source and destination are swapped.

```
vector<uint8_t> buffer;  
size_t offset = 0;  
MyStruct obj;  
std::memcpy(&obj.name, buffer.data() + offset,  
            sizeof(obj.name))  
offset += sizeof(obj.name);  
std::memcpy(&obj.id, buffer.data() + offset,  
            sizeof(obj.id))  
offset += sizeof(obj.id);
```

2.2 Day 2: First Prototype of a Page

A Page in reference to OS, is a kind of unit of storage. Usually when the computer wants to play around with data, it usually does so in the form of Pages.

Throughout this project we will see my page design get updated repeatedly. For now though, our Page is a somewhat simple class as given as below:

```
// struct for acting as a record
struct [[gnu::packed]] UserRecord
{
    uint32_t id;
    char username[32];
    uint32_t age;
}; // [[gnu::packed]] Prevents compiler padding

class Page
{
private:
    // Exactly 4096 bytes matching typical OS page
    // size
    std::array<uint8_t, 4096> data_buffer;

    // Helper to calculate how large a single
    // record is
    const size_t RECORD_SIZE = sizeof(uint32_t) +
        32 + sizeof(uint32_t); // 40 bytes
    const size_t HEADER_SIZE = sizeof(uint32_t);
        // 4 bytes for
        // storing record count

public:
    Page()
    {
        data_buffer.fill(0); // Zero out the page
        // initially
    }
    const uint8_t* ReadRawData() const { return
        data_buffer.data(); }
    uint8_t* WriteRawData() { return data_buffer.
        data(); }
    // Returns how many records are currently
    // stored (read from the header)
```

```

uint32_t GetRecordCount() const; // returns
    number of records currently in Page

// Appends a record to the page if there is
    space
bool AppendRecord(const UserRecord &record);

// Retrieves a record by its slot index inside
    this page
UserRecord GetRecord(uint32_t index) const;
};

```

Although, we will change much functionality of this class in the future, for now it is a good starting point. We have a clear structure for how data is organized within a page, and the methods for reading and writing records are well-defined.

2.3 Day 3 : Connecting to the Disk

As you might have learnt, data is stored in devices such as magnetic disks or hard drives. On a higher level, data is stored in files. We shall treat one file as basically disk. Much like how the read/write head moves across sectors in a disk, our disk manager moves its pointer across the file with the help of `seekg` and `seekp`.

A high level view is given below.

```
class DiskManager
{
private:
    std::fstream db_file;
    std::string filename;

public:
    DiskManager(const std::string &db_filename) :
        filename(db_filename); // open file
    ~DiskManager(); // close file

    void WritePage(uint32_t page_id, const Page &
        page); // write to a page

    void ReadPage(uint32_t page_id, Page &out_page
        ); // read a page
};
```

Obviously you will notice that we don't have a truly complete logic yet. Mainly we aren't tracking the number of pages, nor is there a clear way defined for the bounds of the file.

This will be addressed in Day 4

2.4 Day 4 : Refining the Disk Manager

After looking at the shortcomings of our simple implementation of a disk manager yesterday, we will extend our definition by 2 new helper functions.

```
// Returns the total number of 4KB blocks
// currently inside the file
uint32_t GetTotalPages();
// Grows the file by one blank page and returns
// its new Page ID
uint32_t AllocatePage();
```

This definition will easily suffice for our needs for much of the journey. Much like a physical read/write head, it smoothly scans through our pages, reads the necessary data and writes at the correct locations. Another benefit of this simplistic generic implementation is that we can often change the Page definition multiple times (as we will soon see in the later phases), we hardly have to change the logic of our disk manager. This is the power of Abstraction at play !

2.5 Day 5 : First Query

Although we obviously need to program a looootttt more code for solving any general query in the domain and rules clearly set by our language, but right now, we just want to show how we can use the current data structures for a specific query.

Right now our data structure stores records with 3 fields : username, id and age. We will perform a query equivalent to : Select username, age FROM ... WHERE age $\geq$

The logic is basically to scan every page in our disk, and then scans every record in a page. This type of sequential scan is a precursor to what is about to come :D.

```
void ScanAndPrintUsersOlderThan(DiskManager &disk,
    uint32_t target_age)
{
    uint32_t total_pages = disk.GetTotalPages();
    // YOUR CODE HERE:
    // 1. Loop through every page_id from 0 to
        total_pages - 1
    for (uint32_t i = 0; i < total_pages; i++)
    {
        // 2. Inside the loop, declare a local
            stack Page instance
        Page local_page;
        // 3. Call disk.ReadPage(page_id,
            local_page)
        disk.ReadPage(i, local_page);
        // 4. Get the record count for that page
        uint32_t record_count = local_page.
            GetRecordCount();
        // 5. Loop through every record index in
            that page using local_page.GetRecord(i)
        for (uint32_t j = 0; j < record_count; j
            ++)
        {
            UserRecord record = local_page.
                GetRecord(j);
            // 6. If record.age > target_age,
                print it to the console!
            if (record.age > target_age)
                std::cout << "User_" << record.
                    username << "_with_Age:__" <<
                        record.age << std::endl;
        }
    }
}
```

```
}
```

The above code is our function used to calculate the required result of the query. A more general approach that we will see later, will involve using a template or a sort of virtual class being used as a base, on top of which we can build complex expressions that can cover logical questions such as 'is age $\geq x$?' and perhaps even mathematical evaluations.

2.6 Day 6 : Buffer Pool Manager

Now, imagine everytime we want some specific data , we have to go to the disk every single time. That would be highly resource consuming, and is usually not preferred when we are dealing with huge loads of data. So we need some sort of "buffer" to store frequently accessed data.

If our disk manager handles the structural "ROM" of our logic, then the buffer pool manager handles the structural "RAM". The logic is basically :

1. Check if Page exists in Buffer.
2. If yes, return it and push page to front.
3. Else first check buffer size. If it is at limit, evict the least recently used Page in the buffer.
4. After at least one free space is in the buffer, pull the page from disk and return it.

A compact view of the class is given below.

```
class BufferPoolManager
{
private:
    static constexpr size_t POOL_SIZE = 3; // Our
        RAM can only hold 3 pages at once

    DiskManager &disk_manager;
    std::array<Page, POOL_SIZE> pool; // The
        physical slots (frames) in RAM
    size_t next_free_frame = 0; // Tracks
        where to insert on a cache miss
    // Inside BufferPoolManager:
    // 1. The list stores page_ids. Front is MRU (
        Most Recently Used), Back is LRU.
    std::list<uint32_t> lru_list;

    // 2. The map stores the frame and the
        iterator (the "bookmark")
    struct PageMetadata
    {
        uint32_t frame_id;
        std::list<uint32_t>::iterator
            list_iterator;
        bool is_dirty;
    };
};
```

```

        std::unordered_map<uint32_t, PageMetadata>
            page_table;
        // Now maps page_id to {frame_id,
            iterator_to_list_node}
public:
    uint32_t cache_hits = 0;
    uint32_t cache_misses = 0;

    BufferPoolManager(DiskManager &disk) :
        disk_manager(disk) {}

    void MarkDirty(uint32_t page_id); \\ mark a
        page as dirty to ensure data consistency.
    Page *FetchPage(uint32_t page_id); \\ Page
        ftching logic as given above
};

```

2.7 Day 7 : Chapter's close

For a final test of our logic built so far, we will test writing to disk, using our buffer pool manager for optimal fetching, test our LRU logic, and ensure that changes written to a page on the buffer pool manager are written back to disk.

Thus our final "test" as such is given below.

```
std::string filename = "integration_test.db";
std::remove(filename.c_str());

// 1. Setup: Load a page
{
    DiskManager disk(filename);
    uint32_t p0 = disk.AllocatePage();
    Page p0_data;
    p0_data.AppendRecord(UserRecord{1, "
        InitialData", 20});
    disk.WritePage(p0, p0_data);
}

// 2. Perform operations with Dirty Bit
{
    DiskManager disk(filename);
    BufferPoolManager bpm(disk);

    // Fetch
    Page *p = bpm.FetchPage(0);

    // Modify the page in RAM
    UserRecord updated{1, "UpdatedData", 99};
    p->AppendRecord(updated); // Assume you added
        an Overwrite/Update method to Page
    // CRITICAL: Mark as dirty!
    bpm.MarkDirty(0);

    // Trigger eviction by filling the pool with
        other pages
    bpm.FetchPage(1);
    bpm.FetchPage(2);
    bpm.FetchPage(3); // Page 0 should now be
        evicted and written to disk
}

// 3. Verify Persistence
{
```

```

    DiskManager disk(filename);
    Page p0_final;
    disk.ReadPage(0, p0_final);

    // If the dirty bit worked, this will be 99,
    // not 20!
    assert(p0_final.GetRecord(1).age == 99);
    std::cout << "Dirty_bit_persistence_test_
        passed!" << std::endl;
}

std::remove(filename.c_str());

```

With that our first phase is officially complete :D. Now our next phase will deal with making our current logic more abstract and generalised. We will also need to ensure appropriate logic for maintaining types for our custom SQL language. To do this we will be making a sort of "volcano like model" whose base is our current logic of fetching pages from the buffer / disk. The upper layers will have the responsibility to be able to perform the necessary complex operations on any data that is passed from the base, according to a set of rules , that are designed to always be expandable.

3 Phase 2: The Volcano Model

3.1 Day 1: Schemas

Ahh, one of the most central parts of SQL is the schema. It lets you define your data in a structured manner. To start off phase 2, we define the Schema class, the Column struct and the TypeID enum class. We begin with our types being only 2 : INT32 and VARCHAR. A column is used to store the type, name length and offset within the schema as metadata. A schema is a collection of columns, that basically defines a table for us. Below is given the skeleton of our new code.

```
// Use an enum for performance-critical type
// checking
enum class TypeId {
    INT32,
    VARCHAR
};

struct Column {
    std::string name;
    TypeId type;
    uint32_t length; // Size in bytes (e.g., 4 for
                     // INT32, N for VARCHAR)
    uint32_t offset; // Pre-calculated byte offset
                     // within a tuple, represents the size of all
                     // columns before it
};

class Schema {
private:
    std::vector<Column> columns;
    uint32_t tuple_size = 0; // Total size of a
                             // single row in bytes

public:
    // Explicitly define a schema by passing a
    // list of columns
    Schema(const std::vector<Column>&
           input_columns) : columns(input_columns);
```

```

~Schema() = default;

const std::vector<Column>& GetColumns(); //
    returns columns as a read only value

uint32_t GetTupleSize(); // returns the size
    of the tuple

uint32_t GetColumnCount(); // returns total
    number of columns

const Column& GetColumn(uint32_t col_idx); //
    returns a specific column
};

```

3.2 Day 2 : Terrific Tuples

When we will be fetching data from our Pages , now we cannot pass them as raw bytes for the higher level executors. Instead we actually need a sort of generalised structure that works for any specific type. Enter the tuple class : a model that allows us to fetch data from any Page, check the appropriate type and return the raw data in that type. We also have a struct called as RID, which we use to uniquely identify the location of our Page and the slot of our record. Here, the slot value will be more relevant in the next phase when we update some of our code in phase 1. A basic idea of today's work is as follows :

```
// Uniquely identifies a physical record on disk
struct RID
{
    uint32_t page_id;
    uint32_t slot_num;

    RID() : page_id(0), slot_num(0) {}
    RID(uint32_t page_id, uint32_t slot_num) :
        page_id(page_id), slot_num(slot_num) {}
};

class Tuple
{
private:
    const uint8_t *data_ptr_; // Zero-copy pointer
                             // to the buffer pool page
    uint32_t size_;           // Size of the tuple
                             // in bytes
    RID rid_;                  // Physical location
                             // of this tuple

public:
    Tuple() : data_ptr_(nullptr), size_(0) {}

    // Initialize the tuple with a pointer to the
    // raw page data
    Tuple(const uint8_t *data_ptr, uint32_t size,
          RID rid)
        : data_ptr_(data_ptr), size_(size), rid_(
            rid) {}

    ~Tuple() = default;
```

```

RID GetRID(); // returns rid
const uint8_t *GetData(); // returns ptr to
    start of Page

// Extracts an integer field using the Schema's
    pre-calculated offset
int32_t GetInt32(const Schema &schema,
    uint32_t col_idx);
// Extracts a string field using the Schema's
    pre-calculated offset
std::string GetVarchar(const Schema &schema,
    uint32_t col_idx);
};

```

3.3 Day 3 : The Heart of the Volcano

Today we will be building the virtual classes that power our most core commands. Any kind of SQL program follows the following pipeline: FROM - >ON - >JOIN - >WHERE - >GROUP BY - >HAVING - >SELECT - >DISTINCT - >ORDER BY - >LIMIT / OFFSET

For our project, we will mostly be dealing with :

FROM - >WHERE - >SELECT .

However, the baseline is the same, which is what we have been building towards : Fetch data from pages, strictly maintain schema and build expressions on top of abstract classes. Some of our virtual baseline is given as below :

```
// executor_context.hpp
class ExecutorContext {
private:
    BufferPoolManager &bpm_;

public:
    explicit ExecutorContext(BufferPoolManager &
        bpm) : bpm_(bpm) {}
    ~ExecutorContext() = default;

    // Disallow copying for memory safety
    ExecutorContext(const ExecutorContext&) =
        delete;
    ExecutorContext& operator=(const
        ExecutorContext&) = delete;

    BufferPoolManager* GetBufferPoolManager() {
        return &bpm_; }
};

// abstract_executor.hpp
class AbstractExecutor {
protected:
    ExecutorContext *exec_ctx_;

public:
    explicit AbstractExecutor(ExecutorContext *
        exec_ctx) : exec_ctx_(exec_ctx) {}
    virtual ~AbstractExecutor() = default;

    /**
     * Initializes the executor.
     */
};
```

```

    * Inside this method, internal states (like
      pointers or counters) should be reset.
    */
    virtual void Init() = 0;

    /**
     * Yields the next tuple from this operator.
     * @param tuple The next tuple produced by this
       operator (output parameter)
     * @param rid The physical record identifier of
       the returned tuple (output parameter)
     * @return true if a tuple was successfully
       produced, false if there are no more tuples
       (EOF)
     */
    virtual bool Next(Tuple *tuple, RID *rid) = 0;

    /** @return The schema of the tuples produced
      by this executor */
    virtual const Schema& GetOutputSchema() const
      = 0;
};

// abstract_expression.hpp
class AbstractExpression
{
public:
    virtual ~AbstractExpression() = default;

    // Evaluates the expression against a specific
    physical tuple
    virtual Value Evaluate(const Tuple *tuple,
        const Schema &schema) const = 0;
};

```

One thing you will notice is that our abstractions use something called as a Value. This is a new class defined for ensuring the general nature of our virtual core. A view of the class is given below :

```

class Value {
private:
    TypeId type_;
    union {
        int32_t int32_val;
    };
};

```

```

        std::string varchar_val; // Handled separately
                                due to non-trivial constructor

public:
    Value() : type_(TypeId::INT32), int32_val(0)
        {}
    Value(int32_t val) : type_(TypeId::INT32),
        int32_val(val) {}
    Value(std::string val) : type_(TypeId::VARCHAR
        ), varchar_val(val) {}

    TypeId GetType(); // returns types of the
                      value
    int32_t AsInt32(); // returns integer value
    std::string AsVarchar(); // returns varchar
                          value
};

```

With the core setup, we can create a wide range of expressions to evaluate conditions for our data or simply fetch it in some manner. We will see this in the coming days. For now we simply note our cycle as search : Init - >Next - >Next - >Next till condition evaluates to false. - >Init

3.4 Day 4 : Sequential Scanner

The first executor we will build is a rather simple one. It simply starts at some page, scans every record in that page, goes to next page... until it runs out of pages. Every iteration of Next takes it to the next record. For simplicity, this executor scans page ids in increasing order, ie 0..1..2.. and so on. Our Page class has also been upgraded so that it can work for any generic tuple. Both the classes are given below briefly :

```
// Upgraded Page class
class Page {
private:
    std::array<uint8_t, 4096> data_buffer;
    static constexpr size_t HEADER_SIZE = sizeof(
        uint32_t); // 4 bytes for record count

public:
    Page() {
        data_buffer.fill(0);
    }
    // Standard read/write methods
    const uint8_t* ReadRawData();
    uint8_t* WriteRawData();

    uint32_t GetRecordCount(); // Gets number of
        records

    // Generic append: Accepts a raw pointer to any
        serialized tuple data and its size
    bool AppendRecord(const uint8_t* record_data,
        uint32_t record_size);

    // Zero-Copy Accessor: Returns a direct pointer to
        the starting byte of a slot
    const uint8_t* GetRecordPtr(uint32_t index,
        uint32_t record_size);
};

// SeqScanExecutor
class SeqScanExecutor : public AbstractExecutor {
private:
    Schema schema_;
    uint32_t total_pages_;

    // Iteration state
```

```

uint32_t current_page_id_;
uint32_t current_slot_num_;
uint32_t pages_scanned_;
uint32_t start_page_id_;

public:
    SeqScanExecutor(ExecutorContext *exec_ctx, const
        Schema &schema, uint32_t total_pages, uint32_t
        start_page_id = 0)
        : AbstractExecutor(exec_ctx), schema_(schema),
          total_pages_(total_pages),
          current_page_id_(start_page_id),
          current_slot_num_(0), start_page_id_(
            start_page_id), pages_scanned_(0) {}

    void Init() override; // Set's current_page_id to
                          start_page_id

    bool Next(Tuple *tuple, RID *rid) override; //
        Advances to next record in page or first record
        of next page
        // returns false if no more pages to scan

    const Schema& GetOutputSchema() const override;
};

```

3.5 Day 5 : Abstract Expressions

Our WHERE Clause will be a collection of boolean checks, like age > 18, marks < 100, etc. To evaluate these clauses, we will use a generic expression class.

ColumnValueExpression and ConstantValueExpression are the Operand types. While we will build expressions similar to ComparisonExpression, where we take 2 operands and perform the greater than operation. A view of these classes is given below.

```
// Virtual Base Class
class AbstractExpression
{
public:
    virtual ~AbstractExpression() = default;

    // Evaluates the expression against a specific
    // physical tuple
    virtual Value Evaluate(const Tuple *tuple,
        const Schema &schema) const = 0;
};

// Represents a literal constant value inside a
// query (e.g., the number 21)
class ConstantValueExpression : public
    AbstractExpression
{
private:
    Value val_;

public:
    explicit ConstantValueExpression(Value val) :
        val_(val) {}
    // Returns a constant value
    Value Evaluate(const Tuple *tuple, const
        Schema &schema) const override
};

// Represents a variable reference to a column
// inside a tuple (e.g., table.age)
class ColumnValueExpression : public
    AbstractExpression
{
private:
    uint32_t col_idx_;
```

```

public:
    explicit ColumnValueExpression(uint32_t
        col_idx) : col_idx_(col_idx) {}

    // Fetches a column using the data from tuple
    and the schema for offset
    Value Evaluate(const Tuple *tuple, const
        Schema &schema) const override
};

// Represents a binary comparison operation (For
    today, we will explicitly implement GT '>')
class ComparisonExpression : public
    AbstractExpression
{
private:
    std::unique_ptr<AbstractExpression> left_;
    std::unique_ptr<AbstractExpression> right_;

public:
    ComparisonExpression(std::unique_ptr<
        AbstractExpression> left, std::unique_ptr<
        AbstractExpression> right)
        : left_(std::move(left)), right_(std::move
            (right)) {}

    // Evaluates both children, asserts same type
    values and compares their values, returns
    true if lhs > rhs
    Value Evaluate(const Tuple *tuple, const
        Schema &schema) const override
};

```

3.6 Day 6 : Filtering Fun

Continuing from Day 6, we are building towards our WHERE clause. In the past days we have built both Abstract Expressions and Sequential Scanner. Now we will link both to a single unit. The filter executor calls a child abstract executor, (mostly our sequential scanner) then checks if the current row satisfies the expression given. This expression is the predicate.

A rough view of this new executor class is given below :

```
class FilterExecutor : public AbstractExecutor
{
private:
    std::unique_ptr<AbstractExecutor>
        child_executor_;
    std::unique_ptr<AbstractExpression> predicate_
        ;

public:
    FilterExecutor(ExecutorContext *exec_ctx,
                  std::unique_ptr<AbstractExecutor>
                      child_executor,
                  std::unique_ptr<AbstractExpression>
                      > predicate)
        : AbstractExecutor(exec_ctx),
          child_executor_(std::move(child_executor))
          ,
          predicate_(std::move(predicate)) {}

    // Calls child_executor->Init()
    void Init() override

    // Collects tuple from child executor,
    // evaluates it against predicate, returns if
    // true
    // Else it encounters EOF or tries next row
    bool Next(Tuple *tuple, RID *rid) override

    // A filter doesn't mutate rows; its output
    // schema is identical to its child's schema
    const Schema &GetOutputSchema() const override
};
```

3.7 Day 7 : Pinning it with Projection

Our final Executor for this phase completes the *SELECT ... FROM ... WHERE...* pipeline. It collects the Valid rows and selects the columns asked.

For this it uses:

1. A child executor, often a filter executor.
2. A schema describing the new columns
3. A list of column expressions denoting the selected columns from the input

To store these values, we use a temporary buffer It's outline is given below :

```
class ProjectionExecutor : public AbstractExecutor
{
private:
    std::unique_ptr<AbstractExecutor> child_executor_;
    Schema output_schema_;
    std::vector<std::unique_ptr<AbstractExpression>>
        expressions_;

    // Local workspace buffer to hold the newly
    // materialized tuple data
    std::vector<uint8_t> tuple_buffer_;

public:
    // Set the variables and
    // Resize our local buffer to match the exact byte
    // size of our target output layout
    ProjectionExecutor(ExecutorContext *exec_ctx,
                      std::unique_ptr<
                        AbstractExecutor>
                        child_executor,
                      Schema output_schema,
                      std::vector<std::unique_ptr<
                        AbstractExpression>>
                        expressions)
        : AbstractExecutor(exec_ctx),
          child_executor_(std::move(child_executor)),
          output_schema_(output_schema),
          expressions_(std::move(expressions))
    {
        tuple_buffer_.resize(output_schema_.
                             GetTupleSize());
    }
    // Calls child_executor->Init()
```

```
void Init() override

// Pulls the next row from child to return target
// columns or returns EOF
bool Next(Tuple *tuple, RID *rid) override

// returns target schema for result
const Schema &GetOutputSchema() const override
};
```

3.8 Day 8 : End to End Showcase

Whew, that took too long. Anyways We are gonna be wrapping up this phase by creating a custom schema, selecting target columns, specifying the filter condition and finally verifying our results. A brief overview of the testing pipeline is:

```
void TestEndToEndPipeline()
{
    std::string filename = "end_to_end_db.bin";
    std::remove(filename.c_str());

    DiskManager disk(filename);
    BufferPoolManager bpm(disk);
    ExecutorContext exec_ctx(bpm);

    // 1. Core Source Layout: { id (0), username
    //                          (1), age (2) }
    std::vector<Column> src_cols = {
        {"id", TypeId::INT32, 4, 0},
        {"username", TypeId::VARCHAR, 32, 0},
        {"age", TypeId::INT32, 4, 0}};
    Schema src_schema(src_cols);
    uint32_t tuple_size = src_schema.GetTupleSize();

    // 2. Populate Disk Frame: Create 5 users
    //    spread over 3 pages
    uint32_t p0_id = disk.AllocatePage();
    uint32_t p1_id = disk.AllocatePage();
    uint32_t p2_id = disk.AllocatePage();

    // Page 0: Alice (ID: 1, Age: 25), Bob (ID: 2,
    //          Age: 30)
    Page *p0 = bpm.FetchPage(p0_id);
    std::array<uint8_t, 40> alice{0};
    int32_t id1 = 1;
    int32_t age1 = 25;
    std::memcpy(alice.data() + 0, &id1, 4);
    std::memcpy(alice.data() + 4, "alice", 5);
    std::memcpy(alice.data() + 36, &age1, 4);
    p0->AppendRecord(alice.data(), tuple_size);

    std::array<uint8_t, 40> bob{0};
    int32_t id2 = 2;
    int32_t age2 = 30;
```

```

std::memcpy(bob.data() + 0, &id2, 4);
std::memcpy(bob.data() + 4, "bob", 3);
std::memcpy(bob.data() + 36, &age2, 4);
p0->AppendRecord(bob.data(), tuple_size);
bpm.MarkDirty(p0_id);

// Page 1: Charlie (ID: 3, Age: 35)
Page *p1 = bpm.FetchPage(p1_id);
std::array<uint8_t, 40> charlie{0};
int32_t id3 = 3;
int32_t age3 = 35;
std::memcpy(charlie.data() + 0, &id3, 4);
std::memcpy(charlie.data() + 4, "charlie", 7);
std::memcpy(charlie.data() + 36, &age3, 4);
p1->AppendRecord(charlie.data(), tuple_size);
bpm.MarkDirty(p1_id);

Page *p2 = bpm.FetchPage(p2_id);
std::array<uint8_t, 40> donovan{0};
int32_t id4 = 4;
int32_t age4 = 40;
std::memcpy(donovan.data() + 0, &id4, 4);
std::memcpy(donovan.data() + 4, "donovan", 7);
std::memcpy(donovan.data() + 36, &age4, 4);
p2->AppendRecord(donovan.data(), tuple_size);

std::array<uint8_t, 40> eve{0};
int32_t id5 = 2;
int32_t age5 = 23;
std::memcpy(eve.data() + 0, &id5, 4);
std::memcpy(eve.data() + 4, "eve", 3);
std::memcpy(eve.data() + 36, &age5, 4);
p2->AppendRecord(eve.data(), tuple_size);
bpm.MarkDirty(p2_id);

// 3. TARGET PROJECTION SCHEMA: { age (0), id
(1) }
std::vector<Column> dest_cols = {
    {"age", TypeId::INT32, 4, 0},
    {"id", TypeId::INT32, 4, 0}};
Schema dest_schema(dest_cols);

// =====
// WIRE UP THE VOLCANO PIPELINE TREE
// =====

```

```

// Step A: Instantiate the base sequential
// scanner unique pointer
auto scan_node = std::make_unique<
    SeqScanExecutor>(&exec_ctx, src_schema,
    disk.GetTotalPages());

// Step B: Formulate the filter predicate
// condition (age > 26)
auto col_expr = std::make_unique<
    ColumnValueExpression>(2); // age index
auto const_expr = std::make_unique<
    ConstantValueExpression>(Value(26));
auto filter_predicate = std::make_unique<
    ComparisonExpression>(std::move(col_expr),
    std::move(const_expr));

// Step C: Wrap the scan node inside the
// Filter Executor unique pointer
auto filter_node = std::make_unique<
    FilterExecutor>(&exec_ctx, std::move(
    scan_node), std::move(filter_predicate));

// Step D: Map out the projection remapping
// parameters
std::vector<std::unique_ptr<AbstractExpression
>> proj_exprs;
proj_exprs.push_back(std::make_unique<
    ColumnValueExpression>(2)); // Map source
// age to dest col 0
proj_exprs.push_back(std::make_unique<
    ColumnValueExpression>(0)); // Map source
// id to dest col 1

// Step E: Wrap the filter node inside the
// final root Projection Executor node
// YOUR CODE HERE: Initialize the projection
// plan coordinator node
ProjectionExecutor plan_root(&exec_ctx, std::
    move(filter_node), dest_schema, std::move(
    proj_exprs));

// Initialize root execution pipeline
plan_root.Init();

Tuple t;
RID r;

```

```

// First execution iteration: Should filter
// Alice (25), skip to Bob (30), and drop "
// username"
assert(plan_root.Next(&t, &r) == true);
assert(t.GetInt32(dest_schema, 0) == 30); //
// Bob's Age mapped to index 0
assert(t.GetInt32(dest_schema, 1) == 2); //
// Bob's ID mapped to index 1

// Second execution iteration: Should smoothly
// cross physical disk blocks to yield
// Charlie (35)
assert(plan_root.Next(&t, &r) == true);
assert(t.GetInt32(dest_schema, 0) == 35); //
// Charlie's Age mapped to index 0
assert(t.GetInt32(dest_schema, 1) == 3); //
// Charlie's ID mapped to index 1

// Third execution iteration: Should smoothly
// cross physical disk blocks to yield Donovan
// (40)
assert(plan_root.Next(&t, &r) == true);
assert(t.GetInt32(dest_schema, 0) == 40); //
// Donovan's Age mapped to index 0
assert(t.GetInt32(dest_schema, 1) == 4); //
// Donovan's ID mapped to index 1
// Final call: Plan execution complete (EOF)
assert(plan_root.Next(&t, &r) == false);

std::cout << "DAY_8_END-TO-END_PIPELINE_WORKS_
PERFECTLY!_PHASE_2_COMPLETE!" << std::endl;
std::remove(filename.c_str());
}

```

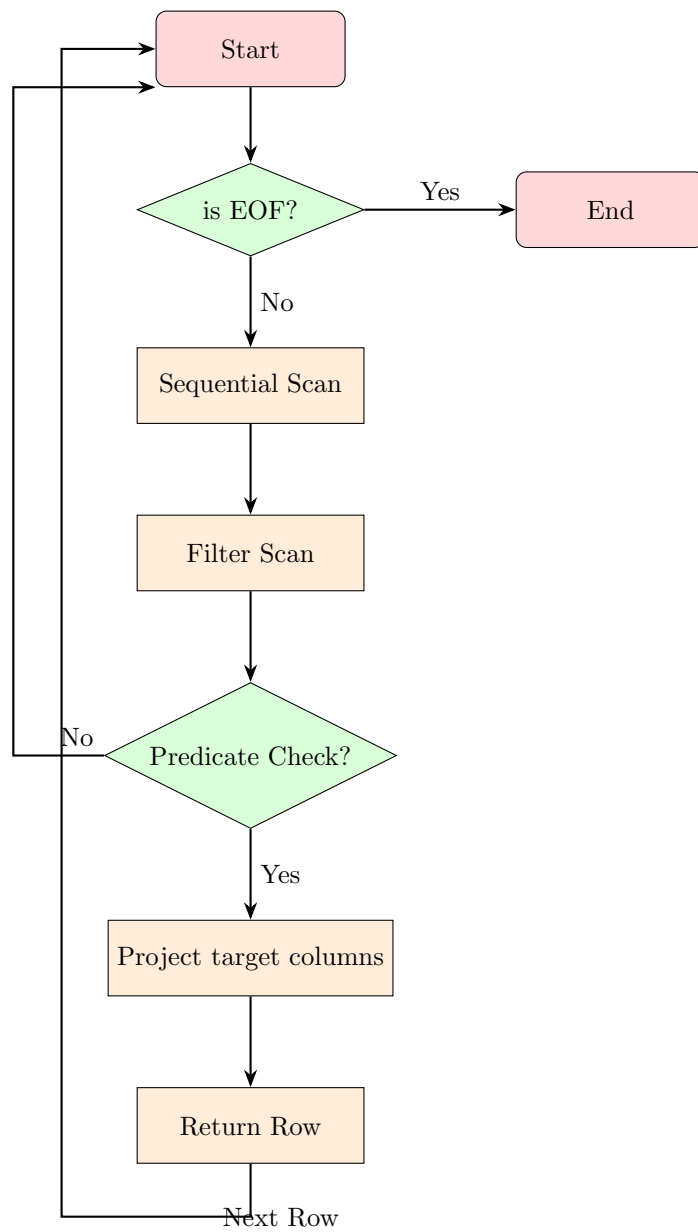


Figure 1: Row scanning and predicate evaluation loop.

4 Phase 3 : The System Catalog

4.1 Day 1 : Table Metadata

Now as we know, each database, has some sort of metadata that helps to make precise queries ahead. That is exactly what this phase aims to tackle. Throughout this phase we will work with a single class *catalog*.

For today we will define two functions :

1. createTable : tableName, schema

2. GetTableSchema : Name

createTable takes in the name of a table, and the schema of the new table. 1st it checks if the table exists, if it does return false and stop. Else create the table metadata. We have a fixed schema structure for table metadata in this form :

```
const Schema tab_schema({
    {const_cast<char *>("name"), TypeId::VARCHAR,
      64, 0, 1},
    {const_cast<char *>("first_page_id"), TypeId::
      INT32, 4, 0, 0},
    {const_cast<char *>("rows"), TypeId::INT32, 4,
      0, 0},
    {const_cast<char *>("schema_page_id"), TypeId
      ::INT32, 4, 0, 0},
    {const_cast<char *>("column_count"), TypeId::
      INT32, 4, 0, 0},
});

const Schema col_schema({
    {const_cast<char *>("name"), TypeId::VARCHAR,
      32, 0, 1},
    {const_cast<char *>("length"), TypeId::INT32,
      4, 0, 0},
    {const_cast<char *>("type"), TypeId::INT32, 4,
      0, 0},
    {const_cast<char *>("is_in_candidate_key"),
      TypeId::INT32, 4, 0, 0},
});
```

GetTableSchema uses our *Filter Executor* from previous phase to search for whether or not the table exists. It then uses the prede-

defined *tab_schema* given above to fetch the table basic data. It uses the *schema_page_id* field to get the schema from there. Our structure is :

1. The whole Database can be considered as a Forest of Linked Lists.
2. The head is Page 0 which has the table Metadata.
3. From the rows of this table, we can get 2 head for each table.
4. One head is the starting page defining the columns of the table.
5. The other head is the starting page of the actual rows of that table.
6. Each list's tail points to *0xFFFFFFFF* as their next page, indicating the end of list.

Tomorrow we'll try for insertion of rows.

4.2 Day 2 : Insert Intelligence

Today we add our new function of *InsertRow*. We also used some helpers like

1. *getTuple* : Validates a list of values against a given schema and returns them in a tuple ready for insertion.
2. *updateRowCount* : Updates the row count in the db metadata and writes to disk, called after write operations that change number of rows.
3. *fetchRowCount* : Fetches the row count from the table metadata.
4. *fetchRowCount_RAW* : Calculates row count by physically reading the disk.

Using these, our insert function inserts a list of given rows. Here atomicity is followed. If even one row is not matching with schema, we reject all. We used an insertion executor for this process. It is similar in nature to the SeqScanExecutor, in how it scans pages. An optional parameter helps count number of inserts, As duplicate candidate key rows are ignored.

```
class InsertionExecutor : public AbstractExecutor
{
private:
    Schema schema_;

    // Iteration state
    uint32_t current_page_id_;
    uint32_t previous_page_id_;
    uint32_t pages_scanned_;
    uint32_t start_page_id_;

public:
    InsertionExecutor(ExecutorContext *exec_ctx, const
        Schema &schema, uint32_t start_page_id = 0,
        uint32_t previous_page_id = 0xFFFFFFFF)
        : AbstractExecutor(exec_ctx), schema_(schema),
          current_page_id_(start_page_id),
          start_page_id_(start_page_id),
          pages_scanned_(0), previous_page_id_(
            previous_page_id) {}

    // Reset execution cursors to point to the very
    // first slot on the first page
    void Init() override;

    // Insert if you can, for now returns false only
    // if the size of the record exceeds the maximum
```

```

        possible slot size in a page
    bool Next(Tuple *tuple, RID *rid) override;

    // schema of insertion
    const Schema &GetOutputSchema() const override;
};

```

Except this time instead of using *GetTuplePtr* to read, it uses *InsertTuple* to write, using our page class. We have to note to mark the pages dirty as we are writing using our BufferPoolManager. Also we borrowed a Query operation from the future to check for duplicate rows. Oopss. Off to Day 3

4.3 Day 3 : Quick Queries

Reaching the halfway point, today we will be implementing the *Query* function. This function needs the table name and columns to select from that table as part of main arguments. Additionally, an abstract expression is accepted for filtering and also a list of names which are to be used in output. By default, all rows are returned and column names are unaltered. We use our full pipeline from phase 2 here, i.e. SeqScan Executor - >Filter Executor - >Projection Executor. To help with formatting, the results are returned as a tuple of Bool (whether query was valid), Schema(of the selected columns renamed) and vector of Tuple (result rows).

We also have some new helper functions:

1. *print_table* : Used to print tables. It uses a given Schema and a list of rows to print the table in a format similar to postgresql/mysql in the terminal.
2. *GetSchemaCols* : It returns the column metadata of a table, given the name. It returns this meta data as a vector of tuples. Using that and our static *col_schema*, we can easily print it with the *print_table* function.
3. *GetDBMeta* : It takes no argument and returns the basic info of tables. The schema of this metadata is the predefined *tab_schema*.
4. *makeCNF*: It is currently used for checking if the row to be inserted exists or not by making a CNF expression.

4.4 Day 4 : Updates Underway

Today we will work on *UpdateRow* function. This function updates specified columns in the given table according to some condition. For simplicity of implementation, we will not allow any changes to columns that are part of the candidate key. To streamline the process we have made a new executor for updates as well, as given below.

```
class UpdateExecutor : public AbstractExecutor
{
private:
    // child is mostly a filter executor
    std::unique_ptr<AbstractExecutor> child_executor_;
    std::map<std::string, Value> col_value_map_;

public:
    UpdateExecutor(ExecutorContext *exec_ctx, std::
        unique_ptr<AbstractExecutor> child_executor,
        std::map<std::string, Value> col_value_map) :
        AbstractExecutor(exec_ctx), child_executor_(std
            ::move(child_executor)), col_value_map_(
            col_value_map) {}

    // Initialize child
    void Init() override

    // Update rows given by child or hit eof
    bool Next(Tuple *tuple, RID *rid) override

    // A update doesn't mutate rows; its output schema
    // is identical to its child's schema
    const Schema &GetOutputSchema() const override
};
```

This executor is also used in certain other places such as updating metadata as well like rowcount. Much like Insert, it is first ensured that the update is valid, before being actually implemented. Also an extra pointer variable/address can be supplied to fetch the count of affected rows.

4.5 Day 5 : Delete Dance

Finally we close out the last functions based on Deleting rows and entire Table. For Deleting a row, we introduce a new executor : *DeleteExecutor* , which functions quite similarly to update. Here after the child executor returns the rows according to the conditions, we use our previously defined delete function from our page class. An overview of this new executor is given below.

```
class DeleteExecutor : public AbstractExecutor
{
private:
    // child, mostly filter executor
    std::unique_ptr<AbstractExecutor> child_executor_;
public:
    DeleteExecutor(ExecutorContext *exec_ctx, std::
        unique_ptr<AbstractExecutor> child_executor) :
        AbstractExecutor(exec_ctx), child_executor_(std
            ::move(child_executor)){}

    // Initialise child
    void Init() override

    // Delete the row given by child
    bool Next(Tuple *tuple, RID *rid) override

    // A delete doesn't mutate rows; its output schema
    // is identical to its child's schema
    const Schema &GetOutputSchema() const override
};
```

On the other hand for deleting table we needed a few adjustments to our Page class. Keeping in line with our tombstone style delete, we add an extra flag in the header which indicates whether it is alive or not. To delete a page, we overwrite the bytes of everything else to default state and change this flag. This way we dont have to fragment memory as such. And these "dead" pages can be reused in our updated *AllocatePage* function from our disk manager. A small snippet of important points below :

```
//new functions in Page class
// sets alive page off (tombstone)
void DeletePage()

// check whether page is alive or not
```

```
bool isAlive()

//caller from disk manager to delete a page
bool DeletePage(uint32_t page_id)

//buffer pool manager caller
// remove from Buffer Pool and call delete on the
// page and its successors
uint32_t DeletePageHelper(uint32_t page_id)

// function in catalog
bool DeleteTable(std::string tableName)
```

4.6 Day 6 : I'm Tired Boss

Finally we reached the end of Phase 3. This was the brain and heart of our project. This will now seamlessly be connected to the parser and frontend that will allow to execute the commands from the terminal ! To recap, in this phase , we :

1. Made the catalog class with support to handle creation and deletion of tables, insertion, updation and deletion of rows, querying the table and some helper functions.
2. Made 3 new executors for insert, update and delete.
3. Updated Page, DiskManager, and BufferPoolManager classes.
4. Conducted extensive testing to the best of my ability to ensure no bugs or vulnerabilities are exploited.